

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100
#define MAX_TOKEN_LEN 256

typedef struct {
    char value[MAX_TOKEN_LEN];
} Token;

void print_tokens(Token tokens[], int count)
{
    printf("\n--- Tokenized Input ---\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: %s\n", i + 1, tokens[i].value);
    }
}

int tokenize(char *input, Token tokens[])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0' && input[i] != '\n') {
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        if (input[i] == '\0' || input[i] == '\n') {
            break;
        }
    }
}
```

```

    if (input[i] == '\\0' || input[i] == '\\n') {
        break;
    }

    if (count >= MAX_TOKENS) {
        printf("Error: Too many tokens.\\n");
        return -1;
    }

    int j = 0;

    while (input[i] != '\\0' &&
           input[i] != '\\n' &&
           !isspace((unsigned char)input[i])) {

        if (j < MAX_TOKEN_LEN - 1) {
            tokens[count].value[j++] = input[i];
        }

        i++;
    }

    tokens[count].value[j] = '\\0';
    count++;
}

return count;
}

```

```

int main()
{
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("myShell> ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 0;
    }

    int count = tokenize(input, tokens);

    if (count == -1) {
        return 1;
    }

    if (count == 0) {
        printf("Empty command.\n");
        return 0;
    }

    print_tokens(tokens, count);

    printf("\n--- Parsing Result ---\n");

    for (int i = 0; i < count; i++) {
        printf("Argument %d: %s\n", i + 1, tokens[i].value);
    }

    printf("\nSyntax validation: Successful\n");
    printf("Execution structure created successfully.\n");

    return 0;
}

```

```
ramya@LAPTOP-OPDAF309: ~$ mkdir SKILL4
ramya@LAPTOP-OPDAF309: ~$ cd SKILL4
ramya@LAPTOP-OPDAF309: ~/SKILL4$ nano task1.c
ramya@LAPTOP-OPDAF309: ~/SKILL4$ gcc task1.c -o task1
ramya@LAPTOP-OPDAF309: ~/SKILL4$ ./task1
myShell> echo Hello World

--- Tokenized Input ---
Token 1: echo
Token 2: Hello
Token 3: World

--- Parsing Result ---
Argument 1: echo
Argument 2: Hello
Argument 3: World

Syntax validation: Successful
Execution structure created successfully.
ramya@LAPTOP-OPDAF309: ~/SKILL4$ ./task1
myShell>
Empty command.
```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100
#define MAX_ARG_LEN 512

int parse_single_quotes(char *input, char args[][MAX_ARG_LEN])
{
    int argc = 0;
    int i = 0;

    while (input[i] != '\0' && input[i] != '\n') {
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        if (input[i] == '\0' || input[i] == '\n') {
            break;
        }

        if (argc >= MAX_ARGS) {
            printf("Too many arguments.\n");
            return -1;
        }

        int j = 0;

        while (input[i] != '\0' &&
            input[i] != '\n' &&
            !isspace((unsigned char)input[i])) {

            if (input[i] == '\\') {

```

```

        i++;

        while (input[i] != '\\' &&
               input[i] != '\0' &&
               input[i] != '\n') {

            if (j < MAX_ARG_LEN - 1) {
                args[argc][j++] = input[i];
            }

            i++;
        }

        if (input[i] != '\\') {
            printf("Syntax Error: Unmatched single quote.\n");
            return -1;
        }

        i++;
    }
    else {

        if (j < MAX_ARG_LEN - 1) {
            args[argc][j++] = input[i];
        }

        i++;
    }
}

args[argc][j] = '\0';
argc++;
}

return argc;

```

```

    }
}

int main()
{
    char input[MAX_INPUT];
    char args[MAX_ARGS][MAX_ARG_LEN];

    printf("myShell> ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 0;
    }

    int argc = parse_single_quotes(input, args);

    if (argc < 0) {
        return 1;
    }

    if (argc == 0) {
        printf("Empty command.\n");
        return 0;
    }

    printf("\nParsing Result:\n");

    for (int i = 0; i < argc; i++) {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    printf("\nSingle quote parsing completed successfully.\n");

    return 0;
}

```

```
ramya@LAPTOP-OPDAF309:~/SKILL4$ nano task2.c
ramya@LAPTOP-OPDAF309:~/SKILL4$ gcc task2.c -o task2
ramya@LAPTOP-OPDAF309:~/SKILL4$ ./task2
myShell> echo Hello

Parsing Result:
Argument 1: echo
Argument 2: Hello

Single quote parsing completed successfully.
ramya@LAPTOP-OPDAF309:~/SKILL4$ ./task2
myShell> echo 'Hello World'

Parsing Result:
Argument 1: echo
Argument 2: Hello World

Single quote parsing completed successfully.
ramya@LAPTOP-OPDAF309:~/SKILL4$ ./task2
myShell> echo 'Hello World
Syntax Error: Unmatched single quote.
```

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100
#define MAX_ARG_LEN 512

void expand_variable(char *output, int *j, const char *name)
{
    const char *value = getenv(name);

    if (value == NULL) {
        return;
    }

    for (int i = 0; value[i] != '\0'; i++) {
        if (*j < MAX_ARG_LEN - 1) {
            output[(*j)++] = value[i];
        }
    }
}

int parse_double_quotes(char *input, char args[][MAX_ARG_LEN])
{
    int argc = 0;
    int i = 0;

    while (input[i] != '\0' && input[i] != '\n') {
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        if (input[i] == '\0' || input[i] == '\n') {
            return argc;
        }
    }
}
```

```

        break;
    }

    if (argc >= MAX_ARGS) {
        printf("Too many arguments.\n");
        return -1;
    }

    int j = 0;

    while (input[i] != '\0' &&
           input[i] != '\n' &&
           !isspace((unsigned char)input[i])) {

        if (input[i] == '"') {

            i++;

            while (input[i] != '"' &&
                   input[i] != '\0' &&
                   input[i] != '\n') {

                if (input[i] == '$') {

                    i++;

                    char variable[100];
                    int k = 0;

                    while ((isalnum((unsigned char)input[i]) ||
                             input[i] == '_') &&
                           k < 99) {

                        variable[k++] = input[i++];
                    }

                    variable[k] = '\0';
                }
            }
        }
    }

```

```

        expand_variable(args[argc], &j, variable);
    }
    else {
        if (j < MAX_ARG_LEN - 1) {
            args[argc][j++] = input[i];
        }

        i++;
    }
}

if (input[i] != '"') {
    printf("Syntax Error: Unmatched double quote.\n");
    return -1;
}

i++;
}
else {
    if (input[i] == '$') {
        i++;

        char variable[100];
        int k = 0;

        while ((isalnum((unsigned char)input[i]) ||
            input[i] == '_') &&
            k < 99) {
            variable[k++] = input[i++];
        }

        variable[k] = '\0';
    }
}

```

```

        expand_variable(args[argc], &j, variable);
    }
    else {
        if (j < MAX_ARG_LEN - 1) {
            args[argc][j++] = input[i];
        }
        i++;
    }
}

args[argc][j] = '\\0';
argc++;
}

return argc;
}

int main()
{
    char input[MAX_INPUT];
    char args[MAX_ARGS][MAX_ARG_LEN];

    printf("myShell> ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 0;
    }

    int argc = parse_double_quotes(input, args);

    if (argc < 0) {
        return 1;
    }
}

```

```

    if (argc == 0) {
        printf("Empty command.\n");
        return 0;
    }

    printf("\n--- Parsed Arguments ---\n");

    for (int i = 0; i < argc; i++) {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    printf("\nDouble quote parsing completed successfully.\n");

    return 0;
}

```

```

ramya@LAPTOP-OPDAF309:~/SKILL4$ nano task3.c
ramya@LAPTOP-OPDAF309:~/SKILL4$ gcc task3.c -o task3
ramya@LAPTOP-OPDAF309:~/SKILL4$ echo $USER
ramya
ramya@LAPTOP-OPDAF309:~/SKILL4$ ./task3
myShell> echo "Hello $USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello ramya

Double quote parsing completed successfully.
ramya@LAPTOP-OPDAF309:~/SKILL4$ ./task3
myShell> echo "Hello World
Syntax Error: Unmatched double quote.

```

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100
#define MAX_ARG_LEN 512

int parse_escape(char *input, char args[][MAX_ARG_LEN])
{
    int argc = 0;
    int i = 0;

    while (input[i] != '\0' && input[i] != '\n') {
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        if (input[i] == '\0' || input[i] == '\n') {
            break;
        }

        if (argc >= MAX_ARGS) {
            printf("Too many arguments.\n");
            return -1;
        }

        int j = 0;

        while (input[i] != '\0' &&
            input[i] != '\n' &&
            (!isspace((unsigned char)input[i]) ||
            input[i - 1] == '\\')) {

            if (input[i] == '\\') {

```

```
        if (input[i] == '\\') {

            i++;

            if (input[i] == '\\0' || input[i] == '\\n') {
                printf("Syntax Error: Trailing escape character.\n");
                return -1;
            }

            if (j < MAX_ARG_LEN - 1) {
                args[argc][j++] = input[i];
            }

            i++;
        }
        else {

            if (j < MAX_ARG_LEN - 1) {
                args[argc][j++] = input[i];
            }

            i++;
        }
    }

    args[argc][j] = '\\0';
    argc++;
}

return argc;
}
```

```
int main()
{
    char input[MAX_INPUT];
    char args[MAX_ARGS][MAX_ARG_LEN];

    printf("myShell> ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 0;
    }

    int argc = parse_escape(input, args);

    if (argc < 0) {
        return 1;
    }

    if (argc == 0) {
        printf("Empty command.\n");
        return 0;
    }

    printf("\n--- Parsed Arguments ---\n");

    for (int i = 0; i < argc; i++) {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    printf("\nEscape sequence parsing completed successfully.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/SKILL4$ nano task4.c
ramya@LAPTOP-OPDAF309:~/SKILL4$ gcc task4.c -o task4
ramya@LAPTOP-OPDAF309:~/SKILL4$ ./task4
myShell> echo Hello\ World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Escape sequence parsing completed successfully.
ramya@LAPTOP-OPDAF309:~/SKILL4$ ./task4
myShell> echo This\ is\ a\ test

--- Parsed Arguments ---
Argument 1: echo
Argument 2: This is a test

Escape sequence parsing completed successfully.
ramya@LAPTOP-OPDAF309:~/SKILL4$ ./task4
myShell> echo Hello\ \$USER

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello $USER

Escape sequence parsing completed successfully.
```

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    int argc;

    printf("myshell$ ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 0;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }

    argc = 0;

    char *token = strtok(input, " \t");

    while (token != NULL && argc < MAX_ARGS - 1) {
        args[argc++] = token;
        token = strtok(NULL, " \t");
    }
}

```

```

}

args[argc] = NULL;

printf("Parent Process PID: %d\n", getpid());

pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {

    printf("Child Process PID: %d\n", getpid());

    execvp(args[0], args);

    perror(args[0]);
    exit(127);
}

printf("Created Child PID: %d\n", pid);

int status;

if (waitpid(pid, &status, 0) == -1) {
    perror("waitpid");
    return 1;
}

if (WIFEXITED(status)) {
    printf("Child exited with status: %d\n",
           WEXITSTATUS(status));
}
else if (WIFSIGNALED(status)) {
    printf("Child terminated by signal: %d\n",
           WTERMSIG(status));
}

return 0;
}

```

```
ramya@LAPTOP-OPDAF309:~/SKILL4$ nano task5.c
ramya@LAPTOP-OPDAF309:~/SKILL4$ gcc task5.c -o task5
ramya@LAPTOP-OPDAF309:~/SKILL4$ ./task5
myshell$ ls
Parent Process PID: 982
Created Child PID: 983
Child Process PID: 983
task1    task2    task3    task4    task5
task1.c  task2.c  task3.c  task4.c  task5.c
Child exited with status: 0
```

```
ramya@LAPTOP-OPDAF309:~/SKILL4$ ./task5
myshell$ ls @l
Parent Process PID: 1030
Created Child PID: 1031
Child Process PID: 1031
ls: cannot access '@l': No such file or directory
Child exited with status: 2
```